Mountain car project

1 Introduction

The environment analyzed in this project is the Mountain Car environment from Gymnasium. In this setup, a car starts at a random position within a valley. The car has three action options : accelerate left, accelerate right, or do nothing. The objective is to reach the peak of the right hill building momentum by oscillating back and forth. Each step incurs a reward of -1, except upon reaching the goal, which grants a reward of 0. The primary challenge of this environment is its sparse reward function. Initially, the car lacks any guidance on reaching the goal. This project will explore strategies to address this challenge.

2 First steps

As observed with the Random Agent in Figure 1, each episode consistently lasts for 200 actions. Since the actions are taken randomly, the task is never completed. The car requires high momentum to reach the goal, which is nearly impossible without coordinated actions.

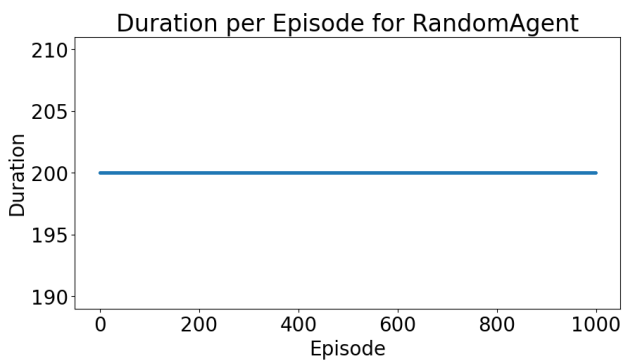

Figure 1 – Scatterplot of duration of each episode for Random Agent

3 Model-free algorithm : Deep Q-Network (DQN)

3.1 No auxiliary reward

The DQN Agent does not select actions randomly. However, due to the sparse rewards, the car fails to learn which actions are beneficial. The average cumulative reward per episode over 1000 episodes is -200, indicating that the agent never reaches the goal even once. The loss curve in Figure 2 shows the loss quickly stabilizing and oscillating around 0.01, which is almost negligible. This happens because the parameters of the MLP converge to predict Q-values of -1, reflecting the reward for every action except reaching the goal.

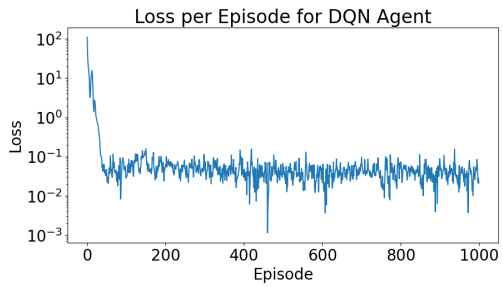

FIGURE 2 – Loss of DQN agent for the first 1000 trials

3.2 Heuristic reward function

We have tried several ideas of auxiliary rewards, experimenting with different definitions of an attractive state. We developed ideas such as rewarding velocity, position, combinations of both, avoiding the "do-nothing" action, giving high termination rewards, and penalizing long duration. Finally, we retained a simple heuristic :

$$\text{Total reward} = r + (x - (-0.5))^2$$

where $r = -1$ is the usual reward given by the environment and x is the position of the car.

However, if the episode is terminated, we give a higher reward that increases inversely to the duration of the episode :

$$\text{Total reward} = r + 1 + 2 \cdot \frac{200 - d}{200}$$

where d is the episode duration and $r = 0$ because the episode has ended.

In our environment, the bottom of the valley is close to -0.5. Therefore, our reward is proportional to the square of the distance to the bottom. This encourages the car to go up to the left and right, consequently learning to gain momentum to reach higher states until it reaches the flag. This quadratic option gives better results than a linear reward because farther states are increasingly more attractive. As the x-axis ranges from -1.2 to 0.5, the distance to -0.5 lies between 0.6 on the left side and 1 on the right side. These values are correctly scaled because our usual rewards are -1 and 0.

Furthermore, the strong positive reward at the end of the episode allows the model to learn the main task. If the episode duration is close to 200 (the maximum), then the reward is just 1, but if it is shorter, e.g., a length of 80, then the car gets 2.2. While this does not allow the car to learn efficiently to minimize its duration each time as this is only an end-of-episode reward, it still helps to reduce it.

We found that an epsilon parameter that decays exponentially from 0.9 to 0.05 in 1700 episodes was optimal. Figure 3 presents the results. The agent solves the problem in 150 episodes. After 3000 trials, the episode duration is either close to 140 actions or 90 actions. These two durations depend on whether the car begins to oscillate on the right or left side. The cumulative number of successes shows an almost parallel curve to the ideal case of solving each time. The loss per episode stabilizes well after 1000 episodes. The two reward curves show that the auxiliary reward is low compared to the negative usual reward but is constant

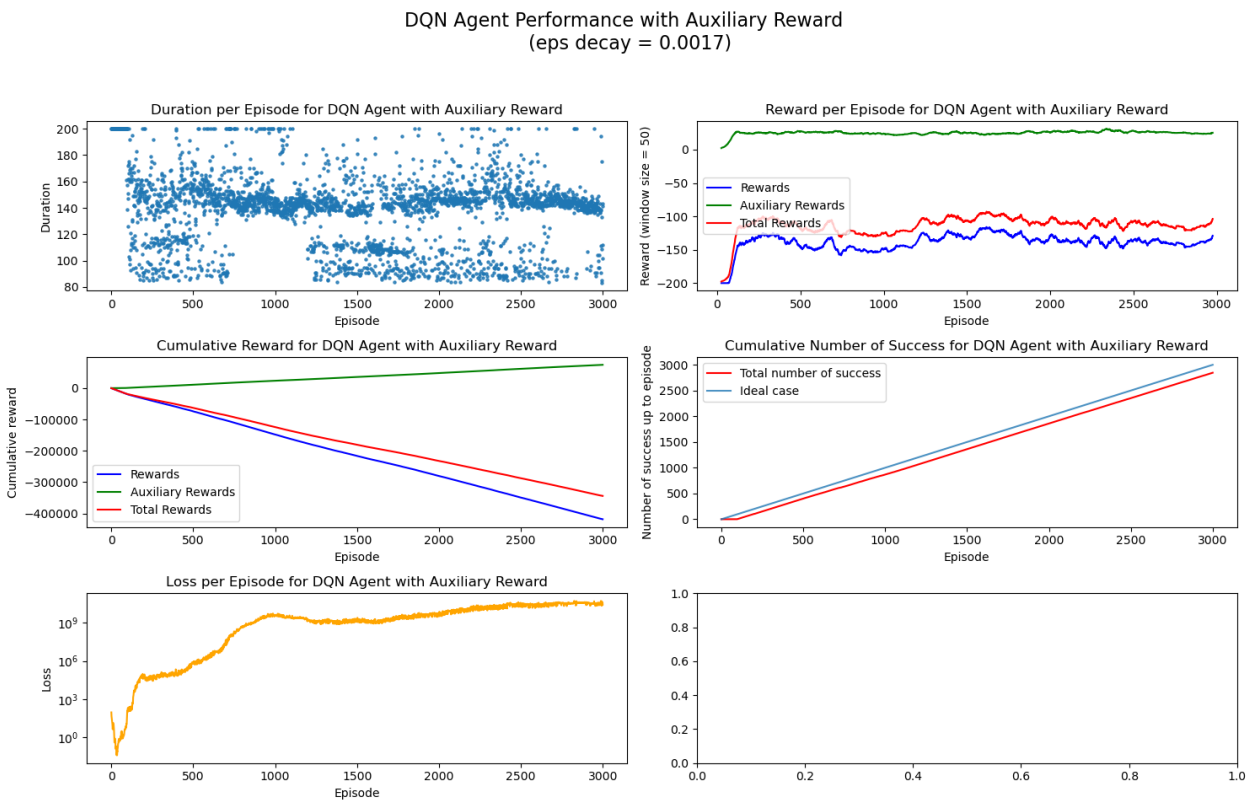

FIGURE 3 – Results for DQN agent with an heuristic reward function

across the episode. This is because the reward depends on the oscillation movement, which is always present, and not on the exploration of new states.

If we choose a very large reward, the learning is faster but more unstable, as new rewards could "overwrite" previous ones and cause the agent to forget. On the contrary, a very low reward leads to slow learning because the agent does not have a strong incentive to take the correct actions since their Q-values are not significantly higher than the others.

3.3 Non domain-specific reward

The two normalizations of the states and the squared difference between the predictor and target are both necessary to keep these numbers on a relevant scale, making the learning more stable. Specifically, standardizing the input to the neural networks ensures consistent and effective learning. The second normalization prevents the intrinsic reward from overshadowing the environment reward or becoming too insignificant.

The reward factor should be set within a range that balances the influence of the intrinsic reward with the environment reward. We chose a value of 0.5. If it is set too high, the intrinsic reward might overshadow the environment reward, leading the agent to focus too much on exploring new states and not enough on achieving the goal. This can result in poor performance on the actual task. Conversely, if the reward factor is set too low, the intrinsic reward might not provide enough incentive for exploration. This can cause the agent to fail to discover enough new states, thereby failing to learn an optimal policy.

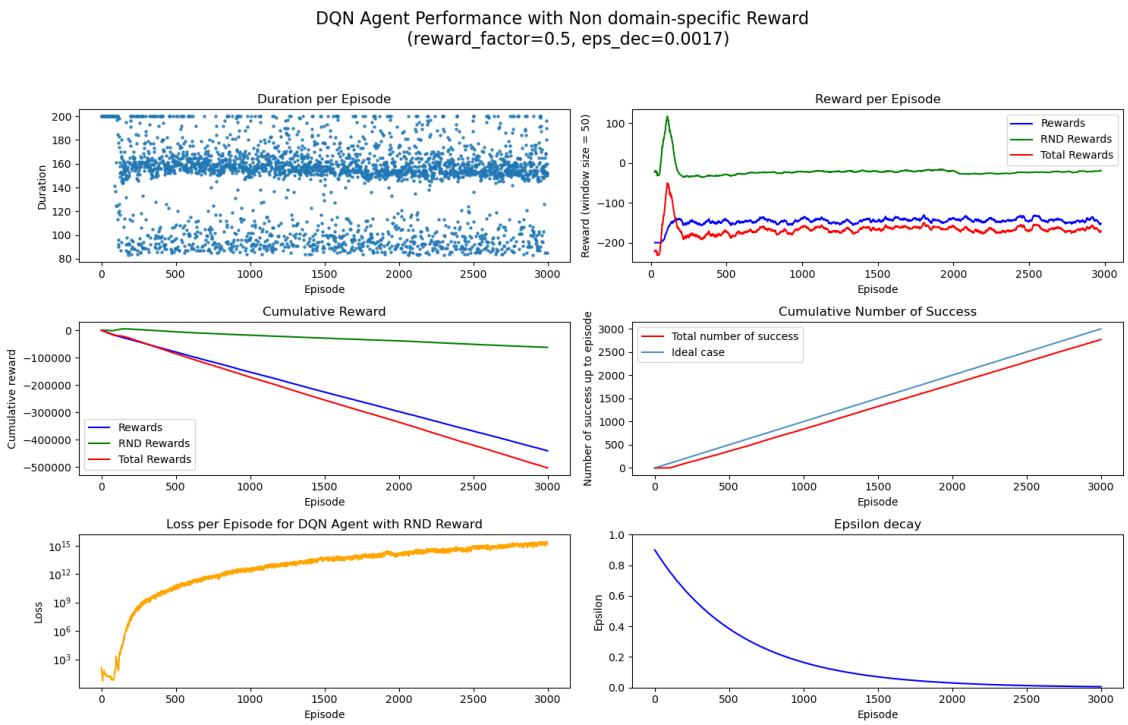

FIGURE 4 – Results for DQN agent with non domain-specific reward

We observe a similar trend as before, where the car reaches the top of the right hill only after 150 episodes, and then it gets better and better over time. We again see many points close to a length of 150 and also close to 90. The cumulative number of successes is also very similar to what we had with the heuristic reward. However, the loss is stabilizing but slower than with the heuristic reward, reaching values of 10^{15}. Additionally, the reward is given in a very different manner, so the two plots about rewards differ significantly. The cumulative reward plot shows a steady decrease for both the environment reward and the total reward, indicating consistent learning. The RND reward remains stable, suggesting it effectively promotes exploration without disrupting learning.

To conclude, the RND reward is more general and promotes exploration by rewarding less frequently visited states. This can lead to more robust exploration but might not be as immediately effective as a well-designed heuristic reward.

Bonus : RND could be considered superior to forward dynamics prediction in stochastic environments because the next state is not deterministic. Therefore, a forward dynamics model might struggle to predict the next state accurately, leading to poor reward signals. Furthermore, RND focuses on the novelty of states rather than trying to predict them accurately, so it is not misled by the inherent unpredictability of the environment. the inherent unpredictability of the environment.

4 Dyna

4.1 Results

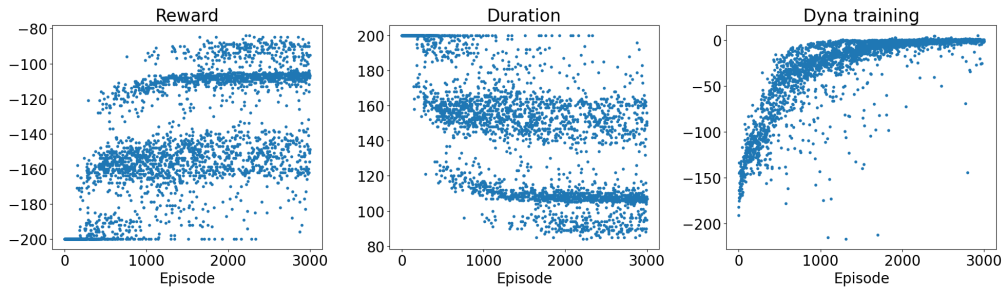

FIGURE 5 – Results for Dyna agent with default bin size

As shown in Figure 5, the Dyna agent solve the task because the episode duration is smaller than 200 iterations. Why does it manages to solve the task even without auxiliary reward ?

The Q-values are initialized to zero while our environment has negative rewards for all states different from the goal state. Hence, while the agent has never reached the goal, all the state values pairs that have been encountered will have a negative Q-value. While the non-encountered state-action pairs will be null, and thus be more favorable. In other words, the agent will tend to avoid all the previous state-action pairs and will keep on exploring the environment to find the new unexplored state-action pairs, until it reaches the goal.

The Dyna agent uses a model of the environment to simulate possible future states and rewards. It can plan ahead and make decisions based on the model's predictions to reach unexplored state-action pairs faster than a model-free agent and thus reaching the goal faster.

When looking at the episode duration plot, we observe patterns emerging as clusters of episode duration. This can be explained from the different and random environment initialization that results with the task being solved with particular patterns, thus taking a particular amount of time.

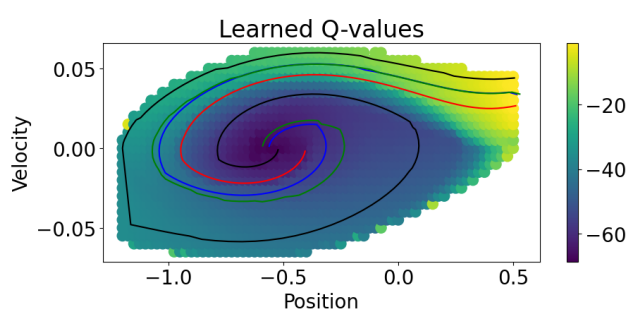

FIGURE 6 – Estimation of Q-values after learning

The leaned Q-values (Figure 6) can give us an intuition of the learned policy. The Q-values are higher in the direction of the goal state which shows that the agent has learned a policy that allows it to reach the goal state. We observe that the agent avoids states with

small absolute position and small velocity, while favoring states with high velocity and high position. This is consistent with the strategy to build up enough momentum to reach the goal. The trajectories line shows a spiral form. The agent starts near the center and start oscillating back on forth more and more strongly until reaching the goal state.

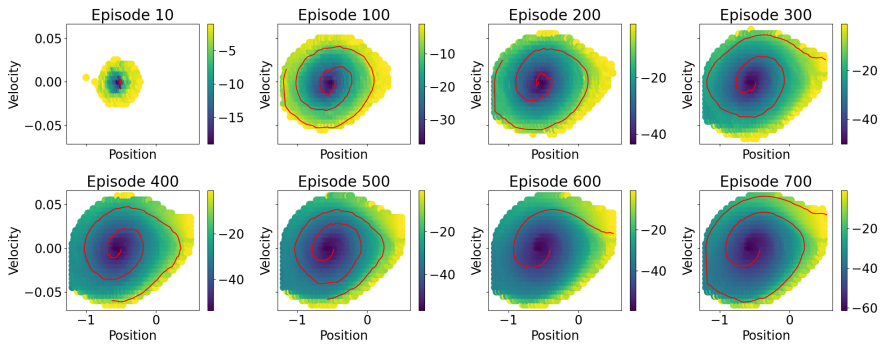

FIGURE 7 – Bonus : Progression of learned Q-values throughout learning

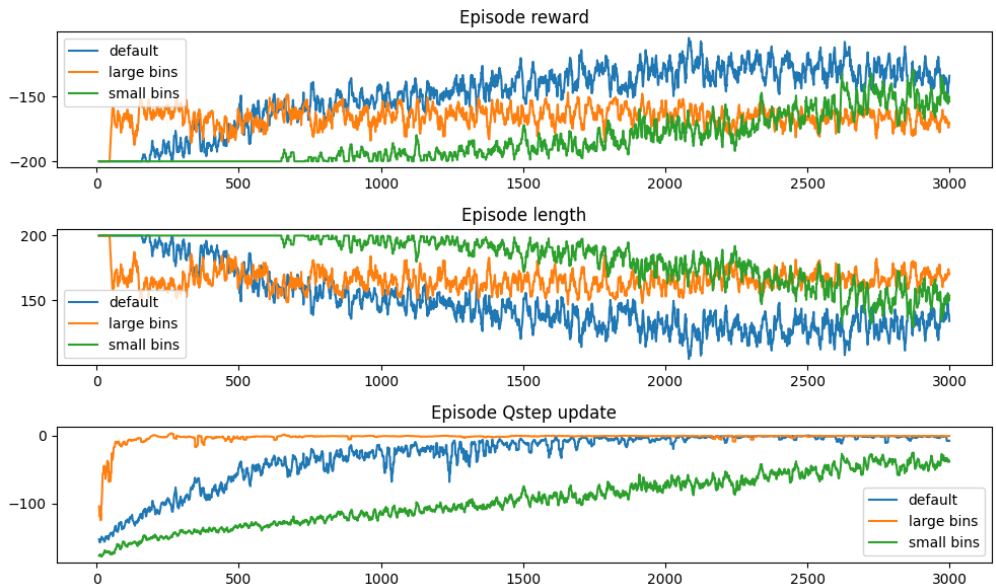

FIGURE 8 – Progression of learned Q-values throughout learning

Overly large bins

Loss of Discriminative Power : While the state space might be smaller, each bin represents a larger region of the original continuous space which makes it more difficult to differentiate between different states within each bin.

Decreased Learning Efficiency : With fewer distinct states, the agent may struggle to learn an effective policy, especially if important distinctions between states are lost. This can result the agent learning sub-optimal policies, hindering the agent's ability to explore the environment efficiently and learn robust policies.

The large bins model solve the learns much faster them the default. It solves the task in the first few iterations which shoes that the agent explored most of the state space. The

Q values converge fast as well, after 700 episodes the Q values update step is close to zero. However, looking at episode rewards shows us that the large bin's model has learned a less good policy.

Overly small bins : With small bins, the state space becomes very large which requires more computational power and memory requirements. This will slow down learning, as it will take much more time for the agent to explore the state space. The small bins model takes around 1600 episode to solve the task for the first time. And we see that the learning still hasn't converged after 3000 episodes as the Q values update step isn't close to zero.

4.2 Comparison with DQN

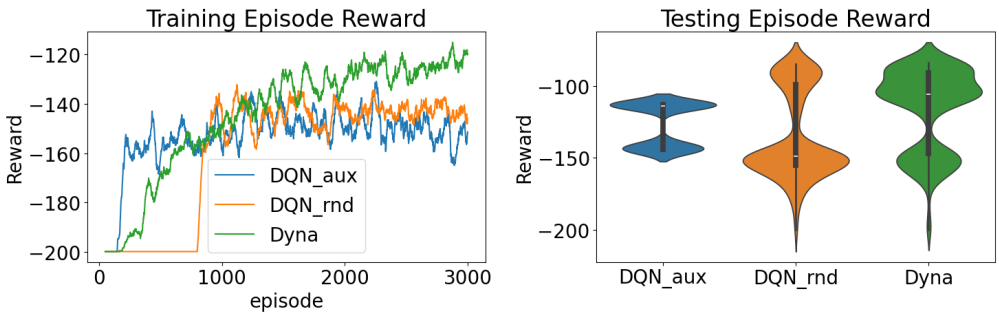

FIGURE 9 – Agent's performance comparison

Agent's episode rewards during training comparison.
As expected, the DQN agent with auxiliary rewards is the first agent to achieve the goal, it is more guided to solve the task. Following the Dyna agent leveraging the internal model representation, and then the DQN agent with non domain specific rewards.

The Dyna agent shows the best performance among the models, achieving the goal faster on average. The DQN agent with auxiliary rewards is the second best, and the DQN agent with non domain specific rewards is the worst. The DQN agent with auxiliary rewards presents less variance in the performance with 2 clear regimes. While the other agents have more variance in the performance.

5 Conclusion

The Dyna agent is model based while the DQN agent is model free. The Dyna agent is able to leverage the internal model representation to achieve better performance, allowing the agent to explore the environment more efficiently and thus reach the goal faster The Dyna agent leverages its internal model representation to simulate the environment which makes is capable of planning ahead and take better decisions. Contrarily to the model-free agent, the Dyna agent learns and thus has knowledge on how to transition between states. The DQN agent will have more trouble going to new states on it's own. Requiring techniques such as hand crafted auxiliary reward or non-domain specific rewards in order to attain the goal effectively.